



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA**

**Complementos de Bases de Datos**

**Sistema de Gestión y Reservas a Medida: Plataforma Web Integral para  
Peluquería y Centro de Estética**

**Realizado por  
Joaquín Borja León  
Ariel Escobar Capilla**

**Departamento  
Lenguajes y Sistemas Informáticos**

**Sevilla, Abril 2026**

---

## Resumen

---

El presente proyecto consiste en el desarrollo de una aplicación web completa diseñada para resolver los problemas de gestión diarios de una peluquería y centro de estética. A diferencia de las soluciones genéricas y costosas del mercado que cobran suscripciones mensuales, esta plataforma está construida a medida para las necesidades exactas de un negocio local.

El sistema se estructura en dos módulos principales interconectados: un portal público de reservas orientado al cliente final, que permite la gestión autónoma de citas las 24 horas del día desde cualquier dispositivo móvil; y un panel de administración privado para los dueños del establecimiento, que proporciona control total sobre la agenda, el historial de clientes, la gestión de horarios y el catálogo de servicios.

Las principales aportaciones del proyecto incluyen: la implementación de un motor de disponibilidad en tiempo real que calcula huecos libres considerando horarios, descansos, bloqueos y citas existentes; un CRM integrado para el seguimiento del historial técnico de cada cliente mediante notas categorizadas; y una arquitectura *single-tenant* que garantiza la privacidad total de los datos del negocio. El desarrollo se realiza utilizando el *stack* MERN (MongoDB, Express, React, Node.js), con especial énfasis en el diseño de una base de datos NoSQL orientada a documentos optimizada para las operaciones del negocio.

**Palabras clave:** Sistema de reservas, aplicación web, gestión de citas, CRM, MongoDB, MERN, barbería, peluquería, *single-tenant*.

---

## Agradecimientos

---

A nuestras profesoras Antonia María Reina Quintero y María Teresa Gómez López, por su orientación y apoyo durante las clases de la asignatura.

A nuestras familias, que regentando su propia peluquería nos trasladaron la idea y la necesidad real que motiva este proyecto. Gracias por compartir con nosotros los retos diarios de la gestión de un negocio local y por su paciencia durante las largas horas de trabajo.

---

## Índice general

---

|                                            |            |
|--------------------------------------------|------------|
| <b>Índice general</b>                      | <b>III</b> |
| <b>Índice de cuadros</b>                   | <b>V</b>   |
| <b>Índice de figuras</b>                   | <b>VI</b>  |
| <b>1 Introducción</b>                      | <b>1</b>   |
| 1.1 Contexto y Motivación . . . . .        | 1          |
| 1.2 Descripción del Problema . . . . .     | 1          |
| 1.3 Objetivos del Proyecto . . . . .       | 2          |
| 1.4 Alcance del Proyecto . . . . .         | 2          |
| 1.5 Tecnologías Utilizadas . . . . .       | 2          |
| <b>2 Diseño de la Base de Datos</b>        | <b>4</b>   |
| 2.1 Justificación de MongoDB . . . . .     | 4          |
| 2.2 Modelo de Datos . . . . .              | 4          |
| 2.2.1 Colección users . . . . .            | 6          |
| 2.2.2 Colección professionals . . . . .    | 6          |
| 2.2.3 Colección services . . . . .         | 7          |
| 2.2.4 Colección appointments . . . . .     | 7          |
| 2.2.5 Colección blockers . . . . .         | 8          |
| 2.2.6 Colección technicalnotes . . . . .   | 9          |
| 2.2.7 Colección settings . . . . .         | 9          |
| 2.3 Estrategia de Indexación . . . . .     | 10         |
| 2.4 Relaciones entre Colecciones . . . . . | 11         |
| <b>3 Implementación</b>                    | <b>12</b>  |
| 3.1 Arquitectura General . . . . .         | 12         |
| 3.2 Implementación del Backend . . . . .   | 13         |
| 3.2.1 Estructura del Proyecto . . . . .    | 13         |
| 3.2.2 API REST . . . . .                   | 13         |
| 3.2.3 Motor de Disponibilidad . . . . .    | 14         |
| 3.2.4 Validación de Datos . . . . .        | 14         |
| 3.3 Implementación del Frontend . . . . .  | 15         |
| 3.3.1 Estructura del Proyecto . . . . .    | 15         |
| 3.3.2 Gestión de Estado . . . . .          | 15         |
| 3.3.3 Sistema de Rutas . . . . .           | 15         |
| 3.3.4 Flujo de Reserva . . . . .           | 15         |
| 3.3.5 Panel de Administración . . . . .    | 16         |
| 3.4 Seguridad . . . . .                    | 16         |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>4</b> | <b>Manual de Usuario</b>                             | <b>17</b> |
| 4.1      | Portal de Reservas (Cliente)                         | 17        |
| 4.1.1    | Registro e Inicio de Sesión                          | 17        |
| 4.1.2    | Proceso de Reserva                                   | 18        |
| 4.1.3    | Gestión de Citas                                     | 20        |
| 4.1.4    | Perfil de Usuario                                    | 21        |
| 4.2      | Panel de Administración                              | 21        |
| 4.2.1    | Dashboard                                            | 21        |
| 4.2.2    | Calendario                                           | 22        |
| 4.2.3    | Gestión de Servicios                                 | 23        |
| 4.2.4    | Gestión de Profesionales                             | 24        |
| 4.2.5    | Gestión de Bloqueos                                  | 25        |
| 4.2.6    | Gestión de Clientes y Notas Técnicas                 | 26        |
| 4.2.7    | Configuración del Negocio                            | 27        |
| <b>5</b> | <b>Manual de Despliegue</b>                          | <b>29</b> |
| 5.1      | Requisitos Previos                                   | 29        |
| 5.2      | Instalación en Entorno Local                         | 29        |
| 5.2.1    | Clonado del Repositorio                              | 29        |
| 5.2.2    | Configuración del Backend                            | 29        |
| 5.2.3    | Configuración del Frontend                           | 30        |
| 5.2.4    | Inicio del Sistema                                   | 30        |
| 5.3      | Despliegue en Producción                             | 30        |
| 5.3.1    | Configuración de MongoDB Atlas                       | 31        |
| 5.3.2    | Despliegue del Backend en Render                     | 31        |
| 5.3.3    | Despliegue del Frontend en Render                    | 31        |
| 5.3.4    | URLs de Despliegue y Consideraciones de Acceso       | 32        |
| <b>6</b> | <b>Conclusiones</b>                                  | <b>33</b> |
| 6.1      | Objetivos Alcanzados                                 | 33        |
| 6.2      | Conocimientos Aplicados                              | 33        |
| 6.3      | Dificultades Encontradas                             | 34        |
| 6.4      | Líneas de Trabajo Futuro                             | 34        |
| 6.5      | Valoración Personal                                  | 34        |
| <b>7</b> | <b>Declaración de Uso de Inteligencia Artificial</b> | <b>35</b> |
|          | <b>Referencias</b>                                   | <b>36</b> |

---

## Índice de cuadros

---

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 1.1 | Tecnologías principales del proyecto . . . . .                   | 3  |
| 2.1 | Estructura de la colección <code>users</code> . . . . .          | 6  |
| 2.2 | Estructura de la colección <code>professionals</code> . . . . .  | 6  |
| 2.3 | Estructura de la colección <code>services</code> . . . . .       | 7  |
| 2.4 | Estructura de la colección <code>appointments</code> . . . . .   | 8  |
| 2.5 | Estructura de la colección <code>blockers</code> . . . . .       | 9  |
| 2.6 | Estructura de la colección <code>technicalnotes</code> . . . . . | 9  |
| 2.7 | Estructura de la colección <code>settings</code> . . . . .       | 10 |
| 2.8 | Índices principales definidos en las colecciones . . . . .       | 10 |
| 3.1 | Principales rutas de la API REST . . . . .                       | 14 |
| 5.1 | Requisitos de software . . . . .                                 | 29 |
| 5.2 | Infraestructura de despliegue en producción . . . . .            | 31 |

---

## Índice de figuras

---

|      |                                                                                  |    |
|------|----------------------------------------------------------------------------------|----|
| 2.1  | Modelo de datos del sistema . . . . .                                            | 5  |
| 3.1  | Arquitectura general del sistema . . . . .                                       | 12 |
| 4.1  | Pantalla de registro de nuevos usuarios. . . . .                                 | 17 |
| 4.2  | Pantalla de inicio de sesión para clientes registrados. . . . .                  | 18 |
| 4.3  | Interfaz principal del portal de clientes tras el inicio de sesión. . . . .      | 18 |
| 4.4  | Paso 1: Selección del servicio a reservar. . . . .                               | 19 |
| 4.5  | Paso 2: Selección de fecha y franja horaria disponible. . . . .                  | 19 |
| 4.6  | Paso 3: Resumen y confirmación final de la reserva. . . . .                      | 20 |
| 4.7  | Sección de gestión de citas del cliente. . . . .                                 | 20 |
| 4.8  | Formulario de confirmación de cancelación de cita. . . . .                       | 21 |
| 4.9  | Pantalla de perfil de usuario. . . . .                                           | 21 |
| 4.10 | Dashboard principal del administrador. . . . .                                   | 22 |
| 4.11 | Vista semanal del calendario de gestión. . . . .                                 | 22 |
| 4.12 | Vista diaria detallada por profesionales (recursos). . . . .                     | 23 |
| 4.13 | Formulario de creación de nueva cita manual por el administrador. . . . .        | 23 |
| 4.14 | Interfaz de gestión del catálogo de servicios. . . . .                           | 24 |
| 4.15 | Formulario de alta de nuevo servicio. . . . .                                    | 24 |
| 4.16 | Gestión de la plantilla de profesionales. . . . .                                | 25 |
| 4.17 | Configuración de un nuevo profesional y sus horarios. . . . .                    | 25 |
| 4.18 | Pantalla de configuración de bloqueos horarios y vacaciones. . . . .             | 26 |
| 4.19 | Formulario de creación de bloqueo de agenda. . . . .                             | 26 |
| 4.20 | Detalle de la ficha de un cliente. . . . .                                       | 27 |
| 4.21 | Formulario para añadir una nueva nota técnica al expediente del cliente. . . . . | 27 |
| 4.22 | Configuración general del establecimiento. . . . .                               | 28 |

---

## Introducción

---

### 1.1– Contexto y Motivación

La digitalización de los pequeños negocios locales se ha convertido en una necesidad en la economía actual. Los salones de belleza, barberías y peluquerías operan mayoritariamente bajo modelos de gestión tradicionales: agenda física, llamadas telefónicas y la memoria del profesional. Sin embargo, los clientes actuales esperan poder reservar en cualquier momento sin realizar llamadas y poder gestionar sus citas fácilmente desde dispositivos móviles.

Este proyecto nace de una necesidad real: la peluquería familiar de uno de nosotros carecía de herramientas digitales adaptadas a su flujo de trabajo. Las soluciones comerciales existentes presentan dos problemas fundamentales: un coste recurrente elevado y una complejidad excesiva con funcionalidades innecesarias que dificultan la adopción por parte de usuarios no técnicos.

La asignatura de Complementos de Bases de Datos ofrece el marco idóneo para abordar este reto, ya que el núcleo del sistema reside en el diseño e implementación de una base de datos que soporte eficientemente todas las operaciones del negocio: gestión de citas con control de solapamientos, historial técnico de clientes, configuración de horarios y bloqueos temporales.

### 1.2– Descripción del Problema

La gestión tradicional de citas en pequeños negocios locales presenta las siguientes deficiencias:

**Pérdida de tiempo productivo:** Las interrupciones por llamadas telefónicas o mensajes de WhatsApp para agendar, modificar o cancelar citas suponen una distracción continua. El profesional debe elegir entre ignorar la llamada o interrumpir el servicio que está prestando, afectando negativamente a la experiencia del cliente actual.

**Descontrol del historial:** Sin un sistema centralizado, resulta difícil recordar qué servicio se le realizó a un cliente hace meses. Información crítica como fórmulas de color, preferencias de corte o alergias se pierde o queda dispersa en notas físicas.

**Falta de métricas:** La ausencia de datos estructurados impide obtener una visión clara del rendimiento del negocio. Preguntas como «¿cuáles son los servicios más demandados?» o «¿cuál es la facturación mensual?» carecen de respuesta inmediata.

**Errores humanos:** La gestión manual es propensa a solapamientos de citas (*overbooking*) o huecos vacíos innecesarios, con impacto económico directo en la rentabilidad del negocio.



### 1.3– Objetivos del Proyecto

El objetivo general es desarrollar una plataforma web integral que digitalice la gestión completa de una barbería o peluquería. Los objetivos específicos son:

1. Diseñar e implementar una base de datos en MongoDB que modele las entidades del dominio: clientes, profesionales, servicios, citas, bloqueos temporales, notas técnicas y configuración del negocio.
2. Desarrollar un portal público de reservas accesible desde dispositivos móviles que permita consultar servicios, visualizar disponibilidad en tiempo real y realizar reservas de forma autónoma.
3. Desarrollar un panel de administración privado con calendario visual interactivo, fichas de cliente con historial técnico (CRM), gestión de horarios por profesional y administración del catálogo de servicios.
4. Implementar un motor de disponibilidad que calcule los huecos libres considerando horarios semanales, descansos, bloqueos y citas existentes, evitando solapamientos.
5. Construir una arquitectura *single-tenant* donde el negocio sea propietario absoluto de sus datos, sin dependencia de plataformas de terceros.

### 1.4– Alcance del Proyecto

El sistema se estructura en dos módulos principales:

**Portal de Reservas (Cliente):** Aplicación web *responsive* donde los clientes pueden explorar el catálogo de servicios con precios y duraciones, ver la disponibilidad en tiempo real, realizar reservas mediante un flujo guiado en tres pasos, y consultar o cancelar sus citas desde un área personal protegida.

**Panel de Administración (Negocio):** Herramienta privada que incluye un *dashboard* con indicadores clave (citas del día, facturación semanal y mensual), un calendario interactivo con vista por profesional basado en FullCalendar, un sistema de fichas de cliente con notas técnicas categorizadas (color, tratamiento, alergia, preferencia), gestión de horarios semanales por profesional con descansos, administración de bloqueos temporales (vacaciones, festivos, mantenimiento) y configuración general del negocio.

**Funcionalidades excluidas del alcance:** Sistema de pagos *online*, marketing automatizado, gestión de inventario, notificaciones por correo electrónico o SMS, y aplicación móvil nativa.

### 1.5– Tecnologías Utilizadas

Para el desarrollo del proyecto se ha seleccionado el *stack* MERN, un conjunto de tecnologías JavaScript ampliamente adoptado en la industria para el desarrollo de aplicaciones web modernas (MongoDB, Inc., 2024a). El servidor se construye con Express.js (OpenJS Foundation, 2024) sobre Node.js, y la interfaz de usuario con React (Meta Platforms, Inc., 2024), estilada mediante Tailwind CSS (Wathan, 2024). Vite (You, 2024) se emplea como herramienta de construcción del *frontend*. El cuadro 1.1 resume las tecnologías principales junto con su versión y propósito.

| Tecnología     | Versión   | Propósito                                       |
|----------------|-----------|-------------------------------------------------|
| MongoDB        | 7.x       | Base de datos NoSQL orientada a documentos      |
| Express.js     | 4.21      | <i>Framework</i> para la API REST del servidor  |
| React          | 19.x      | Biblioteca para la interfaz de usuario          |
| Node.js        | 20.x      | Entorno de ejecución del servidor               |
| Mongoose       | 8.6       | ODM para modelado de datos en MongoDB           |
| Tailwind CSS   | 4.2       | <i>Framework</i> CSS de utilidades              |
| Vite           | 8.0       | Herramienta de construcción del <i>frontend</i> |
| TanStack Query | 5.x       | Gestión de caché y estado del servidor          |
| FullCalendar   | 6.1       | Componente de calendario interactivo            |
| Zustand        | 5.x       | Gestión de estado global ligera                 |
| Zod            | 3.x / 4.x | Validación de esquemas en cliente y servidor    |
| JSON Web Token | 9.x       | Autenticación basada en <i>tokens</i>           |

Cuadro 1.1: Tecnologías principales del proyecto

Adicionalmente, se emplean bibliotecas complementarias como Radix UI para componentes accesibles, Lucide React para iconografía, Day.js y date-fns para el manejo de fechas con soporte de zonas horarias, y Axios como cliente HTTP.

---

### Diseño de la Base de Datos

---

Este capítulo describe el diseño de la capa de persistencia del sistema. Se ha optado por MongoDB, una base de datos NoSQL orientada a documentos, accedida a través de Mongoose como ODM (*Object Document Mapper*). A continuación se justifica la elección, se detallan las colecciones diseñadas y se describe la estrategia de indexación adoptada.

#### 2.1– Justificación de MongoDB

La elección de MongoDB frente a un sistema relacional tradicional se fundamenta en los siguientes criterios:

- **Flexibilidad del esquema:** Las entidades del dominio, como el horario semanal de un profesional, se modelan de forma natural mediante subdocumentos anidados, evitando múltiples tablas auxiliares y *joins* complejos.
- **Cohesión con el *stack*:** MongoDB se integra nativamente con Node.js y Express, formando parte del *stack* MERN. Los documentos JSON almacenados se transfieren directamente entre base de datos, servidor y cliente sin transformaciones intermedias (MongoDB, Inc., 2024c).
- **Rendimiento en lecturas:** Las consultas más frecuentes del sistema (disponibilidad de huecos, listado de citas de un día, fichas de cliente) se resuelven sobre una única colección con índices compuestos, sin necesidad de operaciones de *join*.
- **Escalabilidad horizontal:** Aunque el alcance actual es *single-tenant*, MongoDB facilita el escalado futuro mediante *sharding* y réplicas.

No obstante, se mantiene la integridad referencial a nivel de aplicación mediante Mongoose, que proporciona validaciones de esquema, *hooks* pre/post-guardado y población de referencias (*populate*).

#### 2.2– Modelo de Datos

El sistema se compone de siete colecciones principales, representadas en la figura 2.1. Cada colección se describe a continuación con sus campos, tipos, restricciones y relaciones.

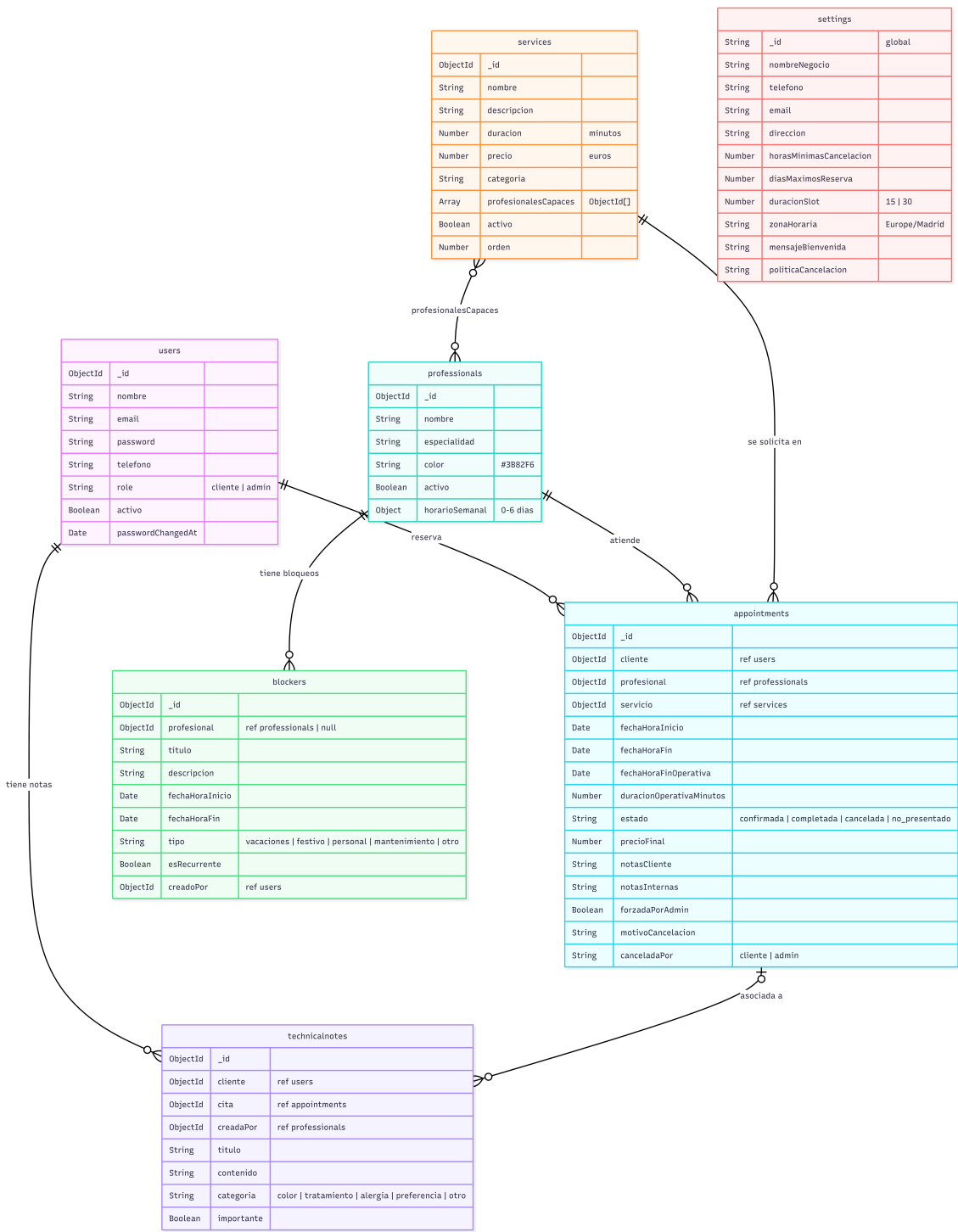


Figura 2.1: Modelo de datos del sistema

### 2.2.1. Colección `users`

Almacena tanto los clientes que realizan reservas como los administradores del sistema. Se distinguen mediante el campo `role`. El cuadro 2.1 detalla su estructura.

| Campo                          | Tipo    | Requerido | Descripción                                                    |
|--------------------------------|---------|-----------|----------------------------------------------------------------|
| <code>nombre</code>            | String  | Sí        | Nombre completo (máx. 100 caracteres)                          |
| <code>email</code>             | String  | Sí        | Correo electrónico, único y en minúsculas                      |
| <code>password</code>          | String  | Sí        | Contraseña cifrada con <code>bcrypt</code> (mín. 6 caracteres) |
| <code>telefono</code>          | String  | No        | Teléfono validado por expresión regular                        |
| <code>role</code>              | String  | Sí        | <code>cliente</code> o <code>admin</code>                      |
| <code>activo</code>            | Boolean | Sí        | Permite desactivar cuentas sin eliminarlas                     |
| <code>passwordChangedAt</code> | Date    | No        | Marca temporal para invalidar <code>tokens</code> anteriores   |

Cuadro 2.1: Estructura de la colección `users`

La contraseña se cifra automáticamente mediante un *hook* `pre-save` de Mongoose con `bcrypt` y un factor de coste de 10. El campo `password` se excluye de las consultas por defecto (`select: false`) para evitar su exposición accidental. Se define un índice sobre el campo `role` para acelerar las consultas de listado de clientes.

### 2.2.2. Colección `professionals`

Representa a los profesionales (peluqueros/barberos) del establecimiento. Cada profesional tiene un horario semanal configurable con soporte para descansos. El cuadro 2.2 muestra los campos principales.

| Campo                       | Tipo    | Requerido | Descripción                                                              |
|-----------------------------|---------|-----------|--------------------------------------------------------------------------|
| <code>nombre</code>         | String  | Sí        | Nombre del profesional (máx. 100)                                        |
| <code>especialidad</code>   | String  | Sí        | Especialidad principal (máx. 100)                                        |
| <code>color</code>          | String  | No        | Color hexadecimal para el calendario (por defecto <code>#3B82F6</code> ) |
| <code>activo</code>         | Boolean | Sí        | Indica si el profesional está disponible                                 |
| <code>horarioSemanal</code> | Object  | Sí        | Subdocumento con la configuración de cada día                            |

Cuadro 2.2: Estructura de la colección `professionals`

El campo `horarioSemanal` es un subdocumento con siete entradas (claves 0 a 6, donde 0 es domingo), cada una con los campos: `activo` (Boolean), `inicio` y `fin` (String en formato HH:MM), y opcionalmente `descansoInicio` y `descansoFin` para la pausa intermedia. Este diseño embebido evita una colección separada de horarios y permite recuperar el horario completo en una sola consulta. El modelo expone métodos de instancia `getHorarioDia(dia)` y `trabajaElDia(dia)` que encapsulan la lógica de acceso.

### 2.2.3. Colección `services`

Define el catálogo de servicios ofrecidos por el establecimiento. El cuadro 2.3 presenta su estructura.

| Campo                | Tipo       | Requerido | Descripción                                         |
|----------------------|------------|-----------|-----------------------------------------------------|
| nombre               | String     | Sí        | Nombre del servicio (máx. 100)                      |
| descripcion          | String     | No        | Descripción detallada (máx. 500)                    |
| duracion             | Number     | Sí        | Duración en minutos (mín. 15)                       |
| precio               | Number     | Sí        | Precio en euros (mín. 0)                            |
| categoria            | String     | No        | Categoría para agrupación (máx. 50)                 |
| profesionalesCapaces | [ObjectId] | No        | Referencias a profesionales que ofrecen el servicio |
| activo               | Boolean    | Sí        | Visibilidad en el catálogo público                  |
| orden                | Number     | No        | Posición para ordenación en la interfaz             |

Cuadro 2.3: Estructura de la colección `services`

El campo `profesionalesCapaces` establece una relación muchos-a-muchos entre servicios y profesionales, almacenada como un *array* de referencias. Esta decisión permite que el motor de disponibilidad filtre rápidamente qué profesionales pueden atender un servicio concreto. El modelo incluye propiedades virtuales `duracionFormateada` y `precioFormateado` que generan cadenas legibles (por ejemplo, 1h 30min o 25,50 €).

### 2.2.4. Colección `appointments`

Es la colección central del sistema, que registra cada cita reservada. El cuadro 2.4 detalla sus campos.

| Campo                    | Tipo     | Requerido | Descripción                                           |
|--------------------------|----------|-----------|-------------------------------------------------------|
| cliente                  | ObjectId | Sí        | Referencia al usuario que reserva                     |
| profesional              | ObjectId | Sí        | Referencia al profesional asignado                    |
| servicio                 | ObjectId | Sí        | Referencia al servicio solicitado                     |
| fechaHoraInicio          | Date     | Sí        | Inicio de la cita (UTC)                               |
| fechaHoraFin             | Date     | Sí        | Fin real de la cita (UTC)                             |
| fechaHoraFinOperativa    | Date     | No        | Fin operativo redondeado a la rejilla de <i>slots</i> |
| duracionOperativaMinutos | Number   | No        | Duración operativa para bloqueo de agenda (mín. 15)   |
| estado                   | String   | Sí        | confirmada, completada, cancelada o no_presentado     |
| precioFinal              | Number   | No        | Instantánea del precio en el momento de la reserva    |
| notasCliente             | String   | No        | Observaciones del cliente (máx. 500)                  |
| notasInternas            | String   | No        | Notas privadas del administrador (máx. 1000)          |
| forzadaPorAdmin          | Boolean  | No        | Indica si se permitió <i>over-booking</i>             |
| motivoCancelacion        | String   | No        | Razón de la cancelación (máx. 500)                    |
| canceladaPor             | String   | No        | cliente o admin                                       |

Cuadro 2.4: Estructura de la colección `appointments`

Se distingue entre `fechaHoraFin` (duración real del servicio) y `fechaHoraFinOperativa` (duración redondeada a la rejilla de 15 o 30 minutos configurada en los ajustes del negocio). Esta distinción permite que un servicio de 20 minutos bloquee un *slot* de 30 minutos en la agenda, evitando fragmentación. El modelo incluye propiedades virtuales `esPasada` y `esHoy` que facilitan el filtrado temporal, y un método `puedeCancelar(horasMinimas)` que verifica si la cita admite cancelación según la política del negocio.

Esta colección cuenta con múltiples índices compuestos, detallados en la sección 2.3.

### 2.2.5. Colección `blockers`

Registra los periodos de tiempo en los que no se pueden agendar citas, ya sea para un profesional concreto o para todo el negocio. El cuadro 2.5 muestra su estructura.

| Campo           | Tipo     | Requerido | Descripción                                         |
|-----------------|----------|-----------|-----------------------------------------------------|
| profesional     | ObjectId | No        | Profesional afectado (null = bloqueo global)        |
| titulo          | String   | Sí        | Título descriptivo (máx. 100)                       |
| descripcion     | String   | No        | Detalle del bloqueo (máx. 500)                      |
| fechaHoraInicio | Date     | Sí        | Inicio del bloqueo (UTC)                            |
| fechaHoraFin    | Date     | Sí        | Fin del bloqueo (debe ser posterior al inicio)      |
| tipo            | String   | Sí        | vacaciones, festivo, personal, mantenimiento u otro |
| esRecurrente    | Boolean  | No        | Reservado para funcionalidad futura                 |
| creadoPor       | ObjectId | No        | Referencia al administrador que lo creó             |

Cuadro 2.5: Estructura de la colección `blockers`

Cuando el campo `profesional` es `null`, el bloqueo se aplica a todo el negocio (por ejemplo, un día festivo). El modelo expone métodos estáticos `hayBloqueo(profesionalId, inicio, fin)` para verificación rápida de solapamientos y `getBloqueos(profesionalId, inicio, fin)` para obtener todos los bloqueos aplicables a un rango, incluyendo los globales.

### 2.2.6. Colección `technicalnotes`

Implementa la funcionalidad de CRM del sistema, permitiendo a los profesionales registrar información técnica sobre cada cliente. El cuadro 2.6 detalla sus campos.

| Campo      | Tipo     | Requerido | Descripción                                     |
|------------|----------|-----------|-------------------------------------------------|
| cliente    | ObjectId | Sí        | Referencia al cliente                           |
| cita       | ObjectId | No        | Referencia a la cita asociada (opcional)        |
| creadaPor  | ObjectId | No        | Referencia al profesional autor                 |
| titulo     | String   | No        | Título de la nota (máx. 100)                    |
| contenido  | String   | Sí        | Contenido de la nota (máx. 2000)                |
| categoria  | String   | No        | color, tratamiento, alergia, preferencia u otro |
| importante | Boolean  | No        | Marca para notas críticas (alergias, etc.)      |

Cuadro 2.6: Estructura de la colección `technicalnotes`

Las categorías permiten clasificar las notas según su naturaleza: fórmulas de *color* aplicadas, *tratamientos* realizados, *alergias* detectadas o *preferencias* del cliente. La marca `importante` permite destacar información crítica que debe consultarse antes de cada servicio.

### 2.2.7. Colección `settings`

Almacena la configuración global del negocio en un único documento con `_id` fijo igual a `global` (patrón *singleton*). El cuadro 2.7 presenta los campos configurables.



| Campo                   | Tipo   | Descripción                                               |
|-------------------------|--------|-----------------------------------------------------------|
| nombreNegocio           | String | Nombre del establecimiento                                |
| telefono                | String | Teléfono de contacto                                      |
| email                   | String | Correo electrónico del negocio                            |
| direccion               | String | Dirección física                                          |
| horasMinimasCancelacion | Number | Horas de antelación mínima para cancelar (por defecto 24) |
| diasMaximosReserva      | Number | Días máximos de antelación para reservar (1–365)          |
| duracionSlot            | Number | Rejilla de <i>slots</i> : 15 o 30 minutos                 |
| zonaHoraria             | String | Zona horaria IANA (por defecto Europe/Madrid)             |
| mensajeBienvenida       | String | Mensaje personalizado en el portal público                |
| politicaCancelacion     | String | Texto de la política de cancelación                       |

Cuadro 2.7: Estructura de la colección `settings`

Este documento se carga en memoria al arrancar la aplicación y se almacena en caché con `Object.freeze()` para garantizar inmutabilidad. Los métodos estáticos `getGlobal()` y `updateGlobal()` gestionan el acceso y la invalidación de la caché respectivamente, evitando consultas repetidas a la base de datos en cada petición.

## 2.3– Estrategia de Indexación

Se han definido índices para optimizar las consultas más frecuentes del sistema. El cuadro 2.8 resume los índices principales.

| Colección      | Índice                                                | Propósito                                     |
|----------------|-------------------------------------------------------|-----------------------------------------------|
| users          | {role: 1}                                             | Filtrar clientes vs. administradores          |
| appointments   | {profesional: 1, fechaHoraInicio: 1}                  | Agenda diaria de un profesional               |
| appointments   | {cliente: 1, fechaHoraInicio: -1}                     | Historial de citas de un cliente              |
| appointments   | {fechaHoraInicio: 1, fechaHoraFin: 1}                 | Consultas de disponibilidad por rango         |
| appointments   | {estado: 1}                                           | Filtrado por estado de la cita                |
| professionals  | {activo: 1}                                           | Listado de profesionales activos              |
| services       | {activo: 1, orden: 1}                                 | Catálogo ordenado de servicios activos        |
| services       | {profesionalesCapaces: 1}                             | Búsqueda de servicios por profesional         |
| blockers       | {profesional: 1, fechaHoraInicio: 1, fechaHoraFin: 1} | Detección de bloqueos por profesional y rango |
| technicalnotes | {cliente: 1, createdAt: -1}                           | Historial de notas de un cliente              |
| technicalnotes | {cliente: 1, importante: 1}                           | Notas críticas de un cliente                  |

Cuadro 2.8: Índices principales definidos en las colecciones

Los índices sobre `appointments` son especialmente críticos, ya que el motor de disponibilidad realiza consultas de rango temporal para cada *slot* calculado. El índice compuesto {`profesional`, `fechaHoraInicio`} permite resolver la consulta de «citas de un profesional en un día concreto» mediante un recorrido secuencial del índice (*index scan*) sin acceder a los documentos.

## 2.4– Relaciones entre Colecciones

Aunque MongoDB no soporta *joins* nativos como los sistemas relacionales, las relaciones entre colecciones se implementan mediante referencias (`ObjectId`) y se resuelven en tiempo de consulta con el método `populate()` de Mongoose (Karpov, 2024). Las relaciones principales son:

- **Appointment** → **User**: Cada cita referencia al cliente que la reservó.
- **Appointment** → **Professional**: Cada cita referencia al profesional asignado.
- **Appointment** → **Service**: Cada cita referencia al servicio solicitado.
- **Service** → **Professional**: Cada servicio contiene un *array* de referencias a los profesionales capacitados para realizarlo (relación muchos-a-muchos).
- **Blocker** → **Professional**: Cada bloqueo puede asociarse a un profesional o ser global.
- **TechnicalNote** → **User**: Cada nota técnica referencia al cliente sobre el que se escribe.
- **TechnicalNote** → **Appointment**: Opcionalmente, una nota puede vincularse a una cita concreta.

La integridad referencial se garantiza a nivel de aplicación: antes de crear una cita, el controlador verifica que el cliente, el profesional y el servicio existen y están activos. Mongoose valida los tipos `ObjectId` en el esquema, rechazando documentos con referencias mal formadas.

## Implementación

Este capítulo describe la arquitectura del sistema y los aspectos más relevantes de su implementación, organizados en tres bloques: el servidor (*backend*), la interfaz de usuario (*frontend*) y los mecanismos transversales de seguridad y sincronización.

### 3.1– Arquitectura General

El sistema sigue una arquitectura cliente-servidor de tres capas, tal como se muestra en la figura 3.1:

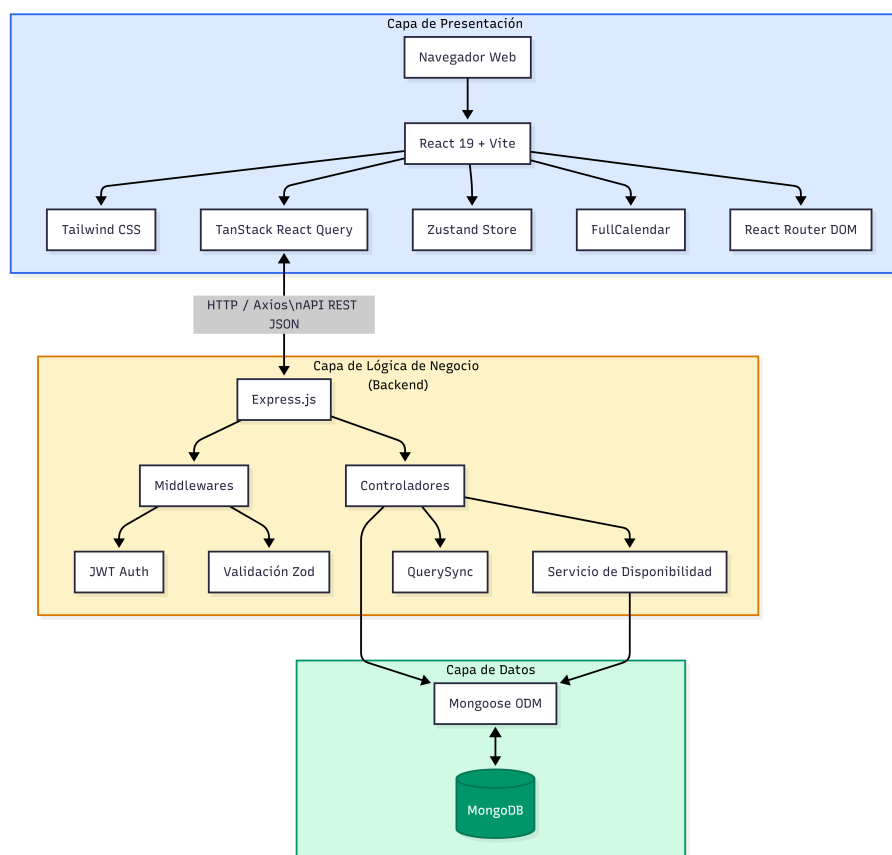


Figura 3.1: Arquitectura general del sistema

1. **Capa de presentación:** Aplicación React servida como SPA (*Single Page Application*) mediante Vite. Se comunica con el servidor exclusivamente a través de peticiones HTTP a la API REST.
2. **Capa de lógica de negocio:** Servidor Node.js con Express que expone una API REST. Contiene los controladores, servicios de disponibilidad, validaciones con Zod y *middleware* de autenticación.
3. **Capa de datos:** Base de datos MongoDB accedida mediante Mongoose. Los esquemas definen validaciones, índices y métodos a nivel de documento.

## 3.2– Implementación del Backend

### 3.2.1. Estructura del Proyecto

El código del servidor se organiza siguiendo el patrón MVC (*Model-View-Controller*) adaptado a una API REST, donde la vista es la respuesta JSON. La estructura de directorios es la siguiente:

- `models/` — Esquemas Mongoose (7 modelos).
- `controllers/` — Lógica de negocio de cada recurso.
- `routes/` — Definición de rutas y asociación con controladores y *middleware*.
- `middlewares/` — Autenticación, autorización, validación y carga de configuración.
- `validators/` — Esquemas Zod para validación de entrada.
- `services/` — Lógica compleja extraída de los controladores (disponibilidad, sincronización de caché).
- `utils/` — Funciones auxiliares de manejo de fechas y zonas horarias.

### 3.2.2. API REST

La API expone nueve grupos de rutas, resumidos en el cuadro 3.1. Todas las rutas están prefijadas con `/api`.

| Recurso        | Método              | Ruta              | Acceso              |
|----------------|---------------------|-------------------|---------------------|
| Autenticación  | POST                | /auth/register    | Público (limitado)  |
|                | POST                | /auth/login       | Público (limitado)  |
|                | GET                 | /auth/me          | Autenticado         |
|                | PUT                 | /auth/me          | Autenticado         |
| Citas          | GET                 | /appointments     | Autenticado         |
|                | POST                | /appointments     | Autenticado         |
|                | PUT                 | /appointments/:id | Admin               |
|                | DELETE              | /appointments/:id | Autenticado         |
| Disponibilidad | GET                 | /availability     | Público             |
| Profesionales  | GET/POST/PUT/DELETE | /professionals    | Admin (GET público) |
| Servicios      | GET/POST/PUT/DELETE | /services         | Admin (GET público) |
| Bloqueos       | GET/POST/PUT/DELETE | /blockers         | Admin               |
| Notas técnicas | GET/POST/PUT/DELETE | /technical-notes  | Admin               |
| Configuración  | GET/PUT             | /settings         | Admin (GET público) |
| Clientes       | GET                 | /auth/clients     | Admin               |

Cuadro 3.1: Principales rutas de la API REST

Cada ruta pasa por una cadena de *middleware* que puede incluir: autenticación (`authenticateToken`), autorización (`requireAdmin`), validación (`validate(schema)`) y carga de ajustes (`loadSettings`).

### 3.2.3. Motor de Disponibilidad

El servicio de disponibilidad (`availability.service.js`) es el componente más complejo del *backend*. Dado un servicio, una fecha y opcionalmente un profesional, calcula los *slots* horarios disponibles. El algoritmo sigue los siguientes pasos:

1. Obtener los profesionales capaces de realizar el servicio solicitado.
2. Para cada profesional, recuperar su horario semanal del día consultado.
3. Generar los *slots* teóricos según la rejilla configurada (15 o 30 minutos), excluyendo los periodos de descanso.
4. Consultar los bloqueos activos (tanto del profesional como globales) y eliminar los *slots* que se solapen.
5. Consultar las citas existentes del profesional en ese día y eliminar los *slots* que se solapen, considerando la duración operativa redondeada.
6. Devolver los *slots* disponibles agrupados por profesional.

Este proceso tiene en cuenta la zona horaria del negocio (configurable en `settings`) mediante las utilidades de `dayjs` con los *plugins* `utc`, `timezone` y `customParseFormat`. Todas las fechas se almacenan en UTC y se convierten a la zona horaria local exclusivamente para el cálculo de *slots* y la presentación al usuario.

### 3.2.4. Validación de Datos

Cada ruta que recibe datos del cliente pasa por un *middleware* de validación basado en Zod (McDonnell, 2024). Los esquemas se definen en archivos dedicados dentro de `validators/` y validan los campos del cuerpo de la petición (`body`), los parámetros de ruta (`params`) y los parámetros de consulta (`query`). Si la validación falla, se devuelve un error 400 con los mensajes de error específicos de cada campo, sin que la petición llegue al controlador.

## 3.3– Implementación del Frontend

### 3.3.1. Estructura del Proyecto

La aplicación cliente se construye con React 19, Vite como *bundler* y Tailwind CSS para los estilos. La estructura de directorios sigue una organización por responsabilidad:

- `pages/` — Componentes de página, cada uno asociado a una ruta.
- `components/` — Componentes reutilizables: `ui/` (botones, formularios, modales basados en Radix UI), `admin/` (componentes del panel de administración), `layout/` (cabecera, pie, *sidebar*).
- `hooks/` — *Custom hooks* que encapsulan las llamadas a la API mediante TanStack React Query.
- `stores/` — Almacenes de estado global con Zustand.
- `lib/` — Configuración de Axios, React Query, notificaciones y utilidades.

### 3.3.2. Gestión de Estado

El estado de la aplicación se gestiona en dos niveles:

**Estado del servidor:** Gestionado por TanStack React Query (Linsley, 2024), que mantiene una caché automática de las respuestas de la API con un tiempo de caducidad configurable (*stale time*). Las mutaciones (creación, actualización, eliminación) invalidan automáticamente las consultas relacionadas mediante un sistema de grupos de sincronización (`querySync`). Esto garantiza que todos los componentes reflejen datos actualizados sin necesidad de recargar la página.

**Estado local:** Gestionado por Zustand, una biblioteca minimalista que almacena dos *stores*: `authStore` (usuario autenticado, *token* JWT, funciones de *login/logout*) y `uiStore` (estado del *sidebar*, modales, tema visual, notificaciones). El `authStore` se persiste automáticamente en `localStorage` para mantener la sesión entre recargas.

### 3.3.3. Sistema de Rutas

La navegación se implementa con React Router DOM, definiendo tres niveles de acceso:

- **Rutas públicas:** Página de inicio, *login* y registro. Accesibles sin autenticación.
- **Rutas protegidas:** Reservas, listado de citas propias y confirmación. Requieren autenticación.
- **Rutas de administración:** *Dashboard*, calendario, gestión de servicios, profesionales, bloques, clientes y ajustes. Requieren rol `admin`.

El componente `ProtectedRoute` actúa como guardia de rutas, verificando el estado de autenticación y el rol del usuario. Si el acceso no está autorizado, redirige a la página de *login* almacenando la ubicación original para redirigir de vuelta tras la autenticación.

### 3.3.4. Flujo de Reserva

El proceso de reserva se implementa como un asistente de tres pasos en la página `BookingPage`:

1. **Selección de servicio:** El cliente explora el catálogo filtrable por categoría, visualizando nombre, duración y precio de cada servicio.

2. **Selección de fecha y hora:** Se presenta un calendario para elegir la fecha y, a continuación, los *slots* disponibles calculados por el motor de disponibilidad. Los *slots* se muestran agrupados por profesional con su color identificativo.
3. **Confirmación:** Se muestra un resumen de la reserva (servicio, profesional, fecha, hora, precio) con un campo opcional para notas. Al confirmar, se crea la cita y se redirige a una página de confirmación.

### 3.3.5. Panel de Administración

El panel de administración se compone de las siguientes secciones:

**Dashboard:** Muestra tarjetas con indicadores clave de rendimiento (KPI): citas del día, citas de la semana, facturación mensual y tasa de conversión (citas completadas sobre el total). Incluye un *widget* de calendario mensual y un listado de las citas más recientes.

**Calendario:** Implementado con FullCalendar (Shaw, 2024), ofrece vistas de mes, semana y día con soporte para vista por recursos (cada profesional es una columna). Permite crear citas manualmente, visualizar bloqueos y consultar los detalles de cada cita mediante un modal.

**Gestión de entidades:** Las páginas de servicios, profesionales, bloqueos y clientes comparten un componente de tabla reutilizable (AdminTable) con funcionalidad de búsqueda, creación, edición y eliminación. La página de profesionales incluye además un editor de horario semanal integrado.

**Configuración:** Permite modificar los parámetros globales del negocio (nombre, contacto, zona horaria, política de cancelación, rejilla de *slots*) desde un formulario centralizado.

## 3.4– Seguridad

La seguridad del sistema se aborda en varias capas:

**Autenticación:** Basada en JSON Web Tokens (JWT) (Jones, Bradley, y Sakimura, 2015) con un periodo de validez de 30 días. El *token* se envía en la cabecera *Authorization* con el esquema *Bearer*. Al cambiar la contraseña, se actualiza el campo *passwordChangedAt* del usuario, invalidando todos los *tokens* emitidos con anterioridad.

**Cifrado de contraseñas:** Las contraseñas se almacenan cifradas con *bcrypt* y un factor de coste de 10 rondas. El campo *password* se excluye de las consultas por defecto para evitar su exposición.

**Limitación de tasa:** Se aplica un límite global de 500 peticiones cada 15 minutos por IP, y un límite específico de 15 peticiones cada 15 minutos en las rutas de autenticación (*login* y registro) para mitigar ataques de fuerza bruta (OWASP Foundation, 2024).

**Cabeceras HTTP:** Se configuran cabeceras de seguridad como *X-Content-Type-Options: nosniff*, *X-Frame-Options: DENY* y se elimina la cabecera *X-Powered-By* para reducir la superficie de ataque.

**Validación de entrada:** Todas las entradas del usuario se validan con esquemas *Zod* antes de llegar al controlador, previniendo inyecciones y datos malformados. El tamaño del cuerpo de las peticiones se limita a 10 KB.

**CORS:** Se configura una lista blanca de orígenes permitidos, restringiendo en producción las peticiones al dominio del *frontend*.

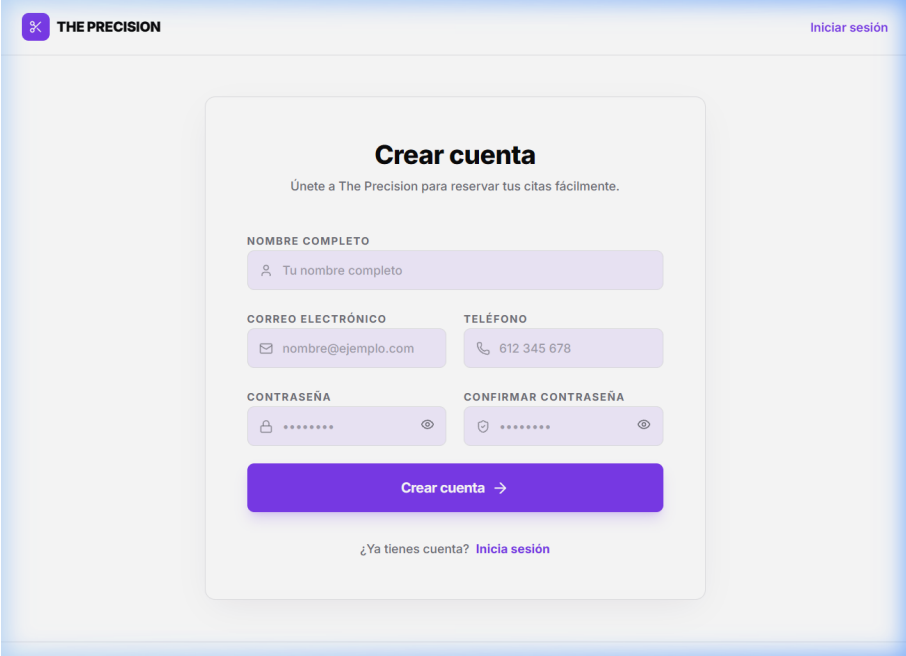
## Manual de Usuario

Este capítulo describe el uso del sistema desde la perspectiva de los dos tipos de usuario: el cliente que realiza reservas y el administrador que gestiona el negocio.

### 4.1– Portal de Reservas (Cliente)

#### 4.1.1. Registro e Inicio de Sesión

Para utilizar el sistema de reservas, el cliente debe crear una cuenta accediendo a la página de registro (Figura 4.1). Se solicitan los siguientes datos: nombre completo, correo electrónico, contraseña y teléfono de contacto.



La imagen muestra una interfaz web para crear una cuenta. En la parte superior izquierda hay un logo con un icono de tijeras y el texto 'THE PRECISION'. En la parte superior derecha hay un enlace 'Iniciar sesión'. El título principal es 'Crear cuenta' con el subtítulo 'Únete a The Precision para reservar tus citas fácilmente.'.

El formulario contiene los siguientes campos:

- NOMBRE COMPLETO:** Un campo de texto con un ícono de persona y el placeholder 'Tu nombre completo'.
- CORREO ELECTRÓNICO:** Un campo de texto con un ícono de correo y el placeholder 'nombre@ejemplo.com'.
- TELÉFONO:** Un campo de texto con un ícono de teléfono y el placeholder '612 345 678'.
- CONTRASEÑA:** Un campo de texto con un ícono de contraseña y el placeholder '\*\*\*\*\*'.
- CONFIRMAR CONTRASEÑA:** Un campo de texto con un ícono de contraseña y el placeholder '\*\*\*\*\*'.

Debajo de los campos hay un botón azul con el texto 'Crear cuenta →'. En la parte inferior del formulario hay un enlace '¿Ya tienes cuenta? Inicia sesión'.

Figura 4.1: Pantalla de registro de nuevos usuarios.

Para sesiones posteriores, el cliente accede mediante la página de *login* (Figura 4.2) introduciendo sus credenciales.



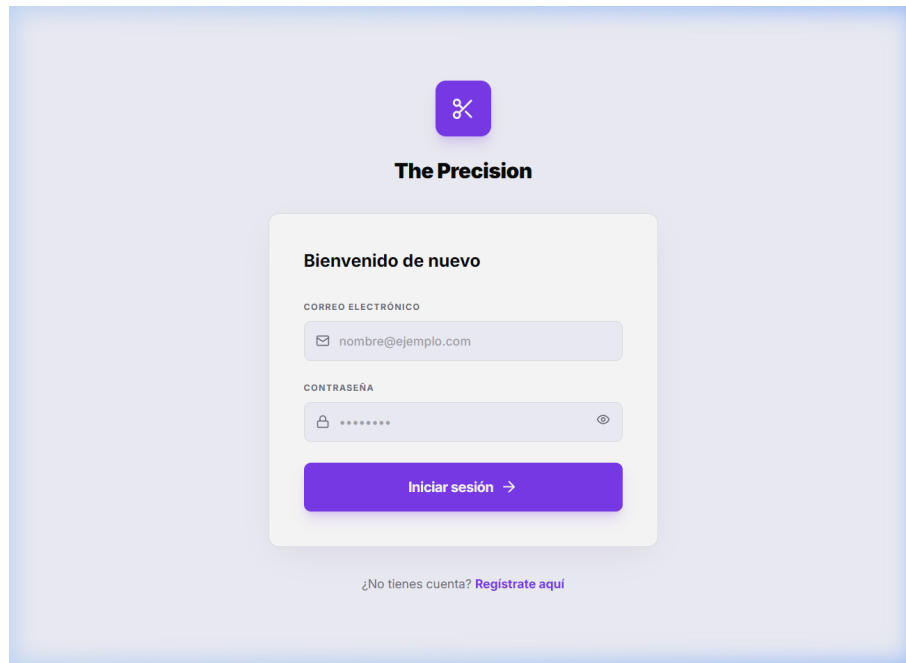


Figura 4.2: Pantalla de inicio de sesión para clientes registrados.

Tras autenticarse, el sistema muestra la pantalla principal del portal (Figura 4.3), donde el usuario puede ver un resumen de su actividad.

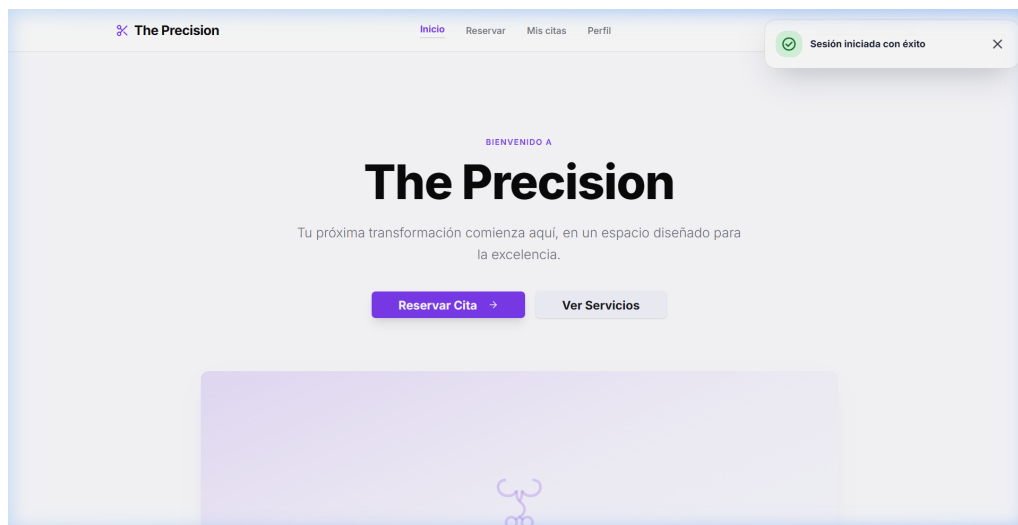


Figura 4.3: Interfaz principal del portal de clientes tras el inicio de sesión.

#### 4.1.2. Proceso de Reserva

La reserva de una cita se realiza en tres pasos guiados:

**Paso 1 — Selección de servicio:** Se muestra el catálogo de servicios (Figura 4.4). El cliente selecciona el servicio deseado.

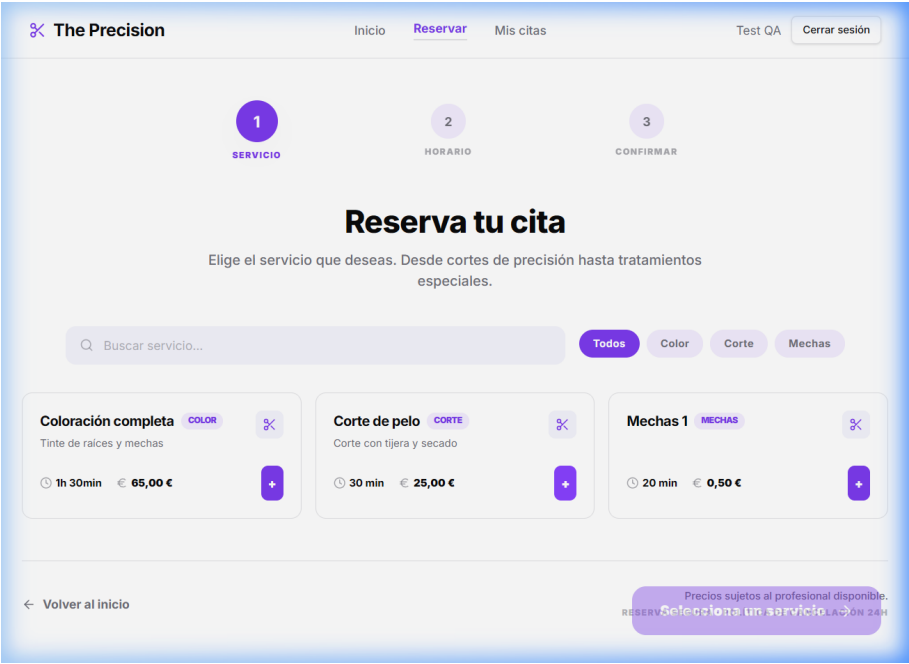


Figura 4.4: Paso 1: Selección del servicio a reservar.

**Paso 2 — Selección de fecha y hora:** Se presenta un calendario interactivo (Figura 4.5) donde se muestran los huecos disponibles calculados en tiempo real.

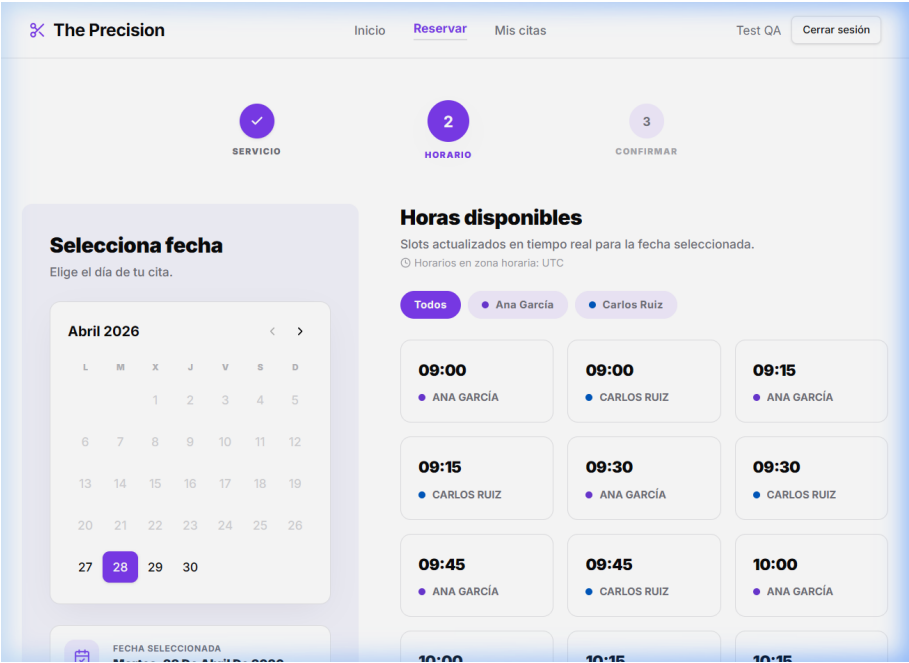


Figura 4.5: Paso 2: Selección de fecha y franja horaria disponible.

**Paso 3 — Confirmación:** Se muestra un resumen final (Figura 4.6) para validar los datos antes de confirmar la reserva.

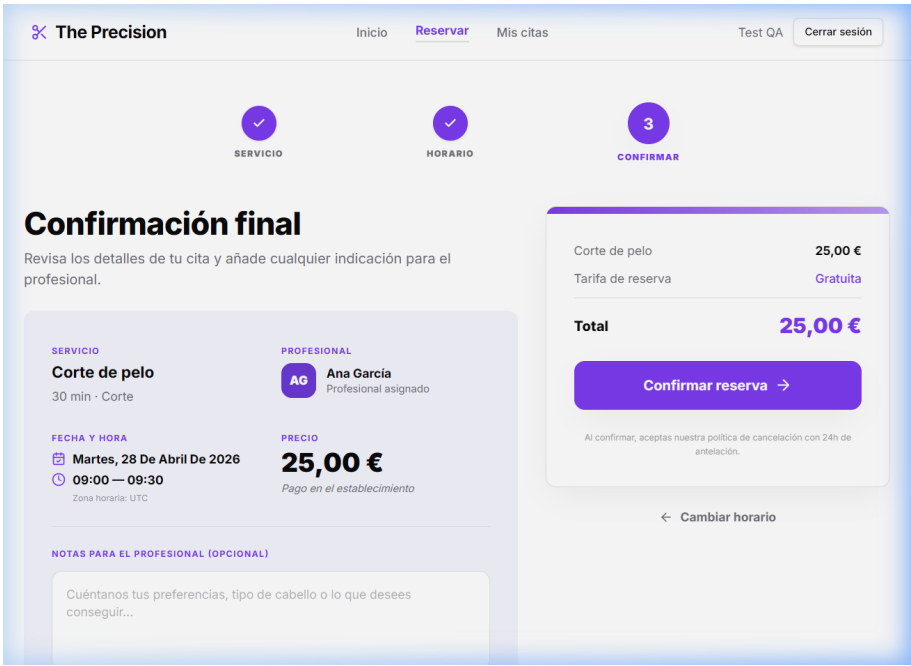


Figura 4.6: Paso 3: Resumen y confirmación final de la reserva.

### 4.1.3. Gestión de Citas

Desde la sección «Mis Citas» (Figura 4.7), el cliente puede consultar el historial de sus citas y cancelar aquellas que estén pendientes mediante un formulario de confirmación (Figura 4.8).

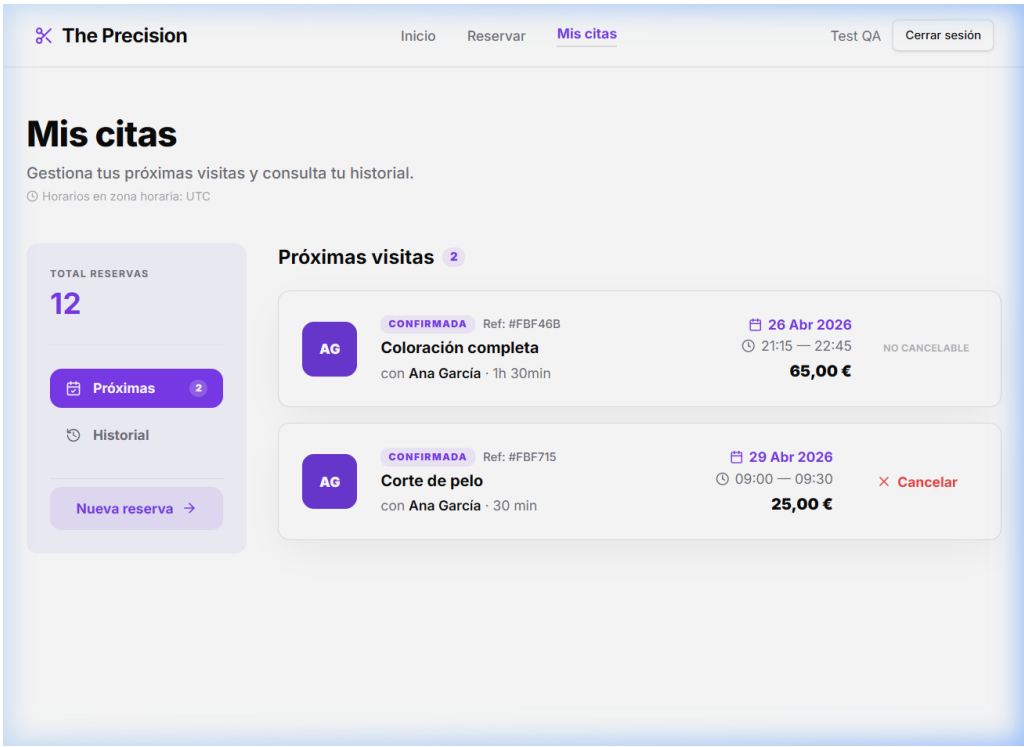


Figura 4.7: Sección de gestión de citas del cliente.

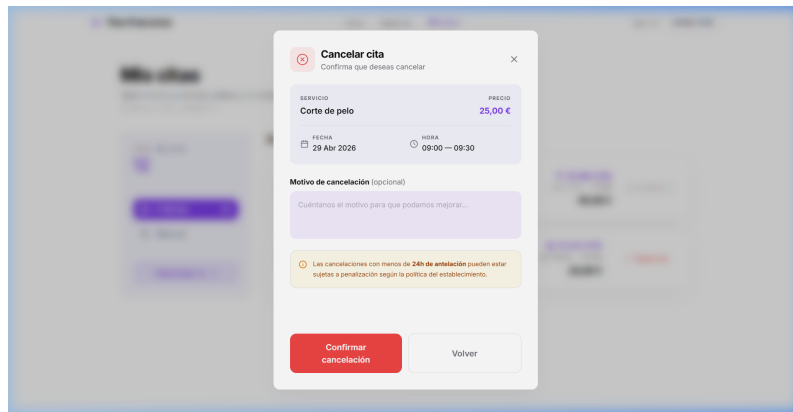


Figura 4.8: Formulario de confirmación de cancelación de cita.

#### 4.1.4. Perfil de Usuario

El cliente puede actualizar sus datos personales y preferencias desde el área de perfil (Figura 4.9).

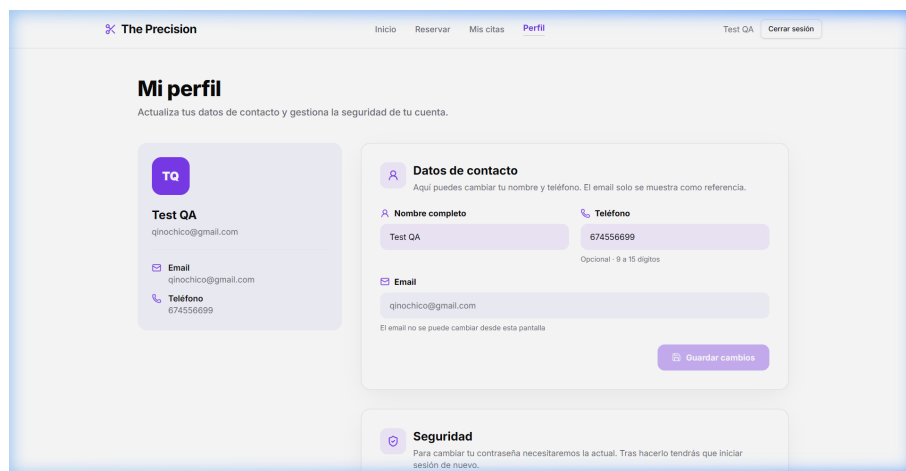


Figura 4.9: Pantalla de perfil de usuario.

## 4.2– Panel de Administración

El panel de administración permite una gestión integral del negocio.

### 4.2.1. Dashboard

El Dashboard (Figura 4.10) ofrece una visión global con indicadores clave y las citas próximas.

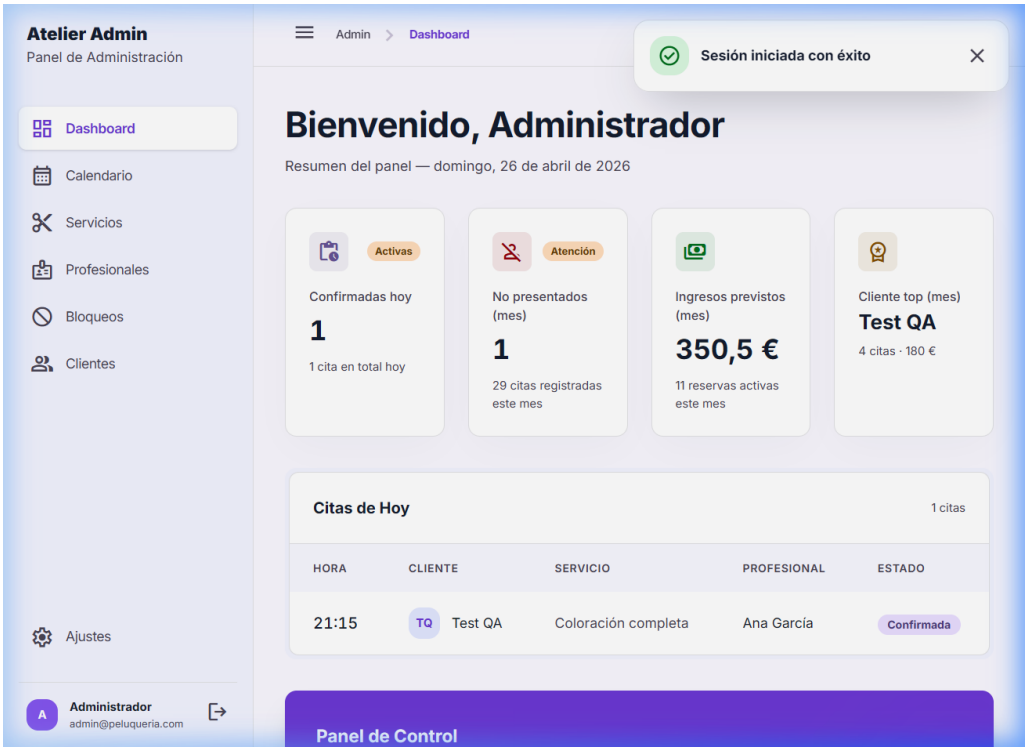


Figura 4.10: Dashboard principal del administrador.

4.2.2. Calendario

El calendario permite visualizar la agenda en diferentes formatos. La vista semanal (Figura 4.11) ofrece una perspectiva amplia, mientras que la vista diaria por recursos (Figura 4.12) facilita la gestión de los profesionales.

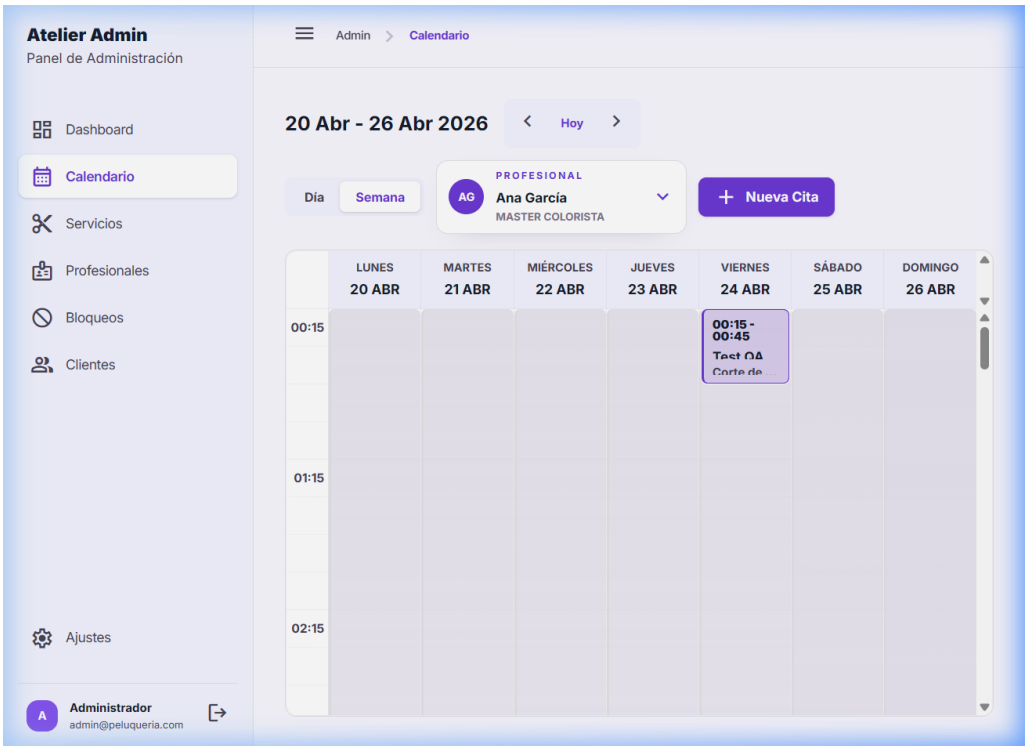


Figura 4.11: Vista semanal del calendario de gestión.

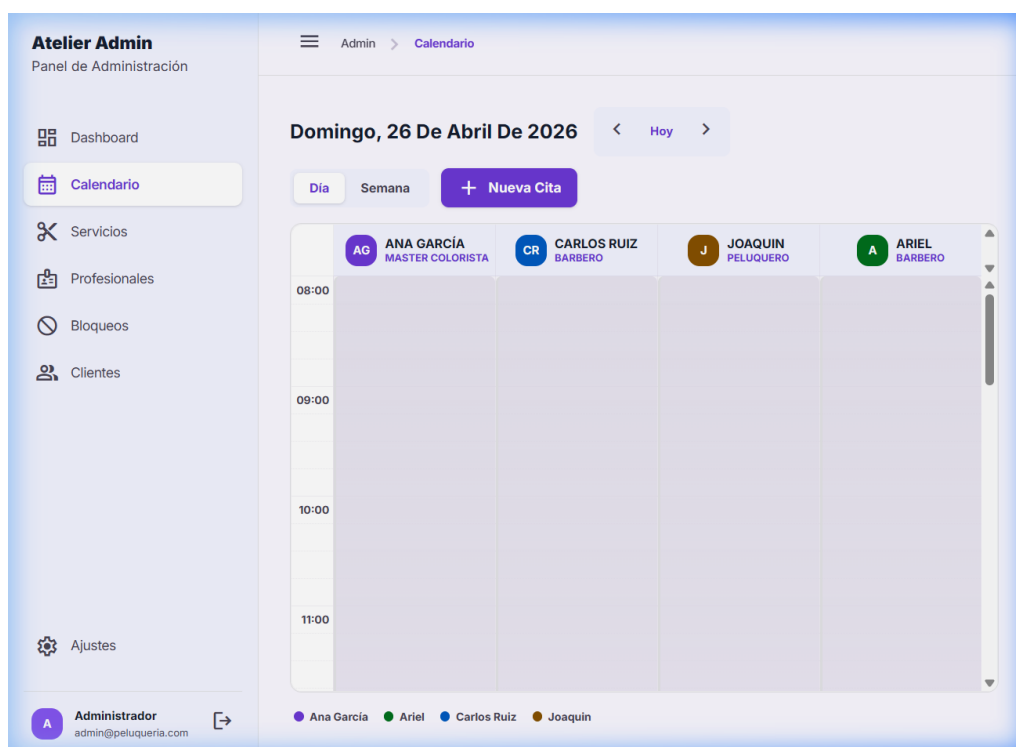


Figura 4.12: Vista diaria detallada por profesionales (recursos).

Desde cualquier vista del calendario, el administrador puede crear nuevas citas manualmente (Figura 4.13).

Figura 4.13: Formulario de creación de nueva cita manual por el administrador.

### 4.2.3. Gestión de Servicios

Permite administrar el catálogo de servicios ofrecidos (Figura 4.14) y dar de alta nuevas opciones (Figura 4.15).

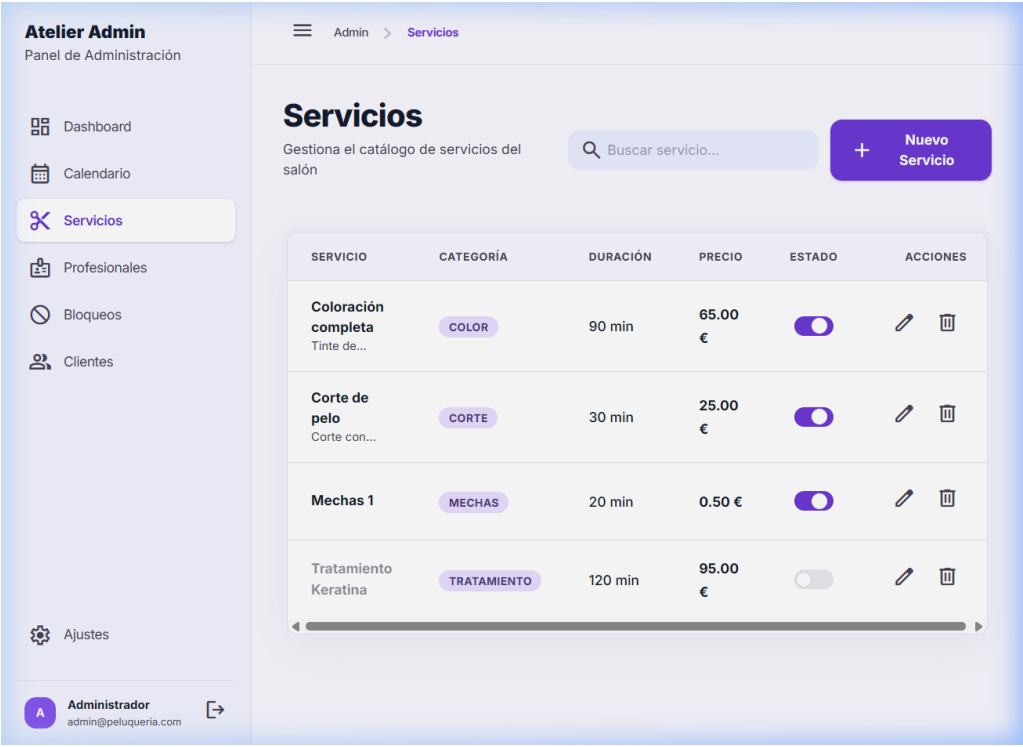


Figura 4.14: Interfaz de gestión del catálogo de servicios.

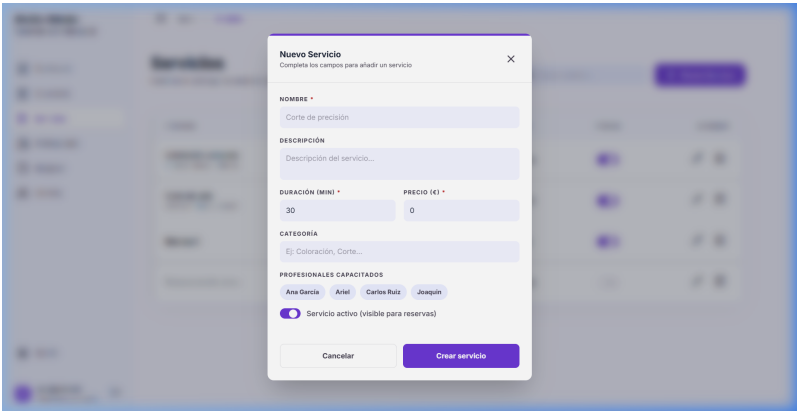


Figura 4.15: Formulario de alta de nuevo servicio.

4.2.4. Gestión de Profesionales

Sección para gestionar el equipo, sus horarios y especialidades (Figura 4.16). Es posible añadir nuevos miembros a la plantilla (Figura 4.17).

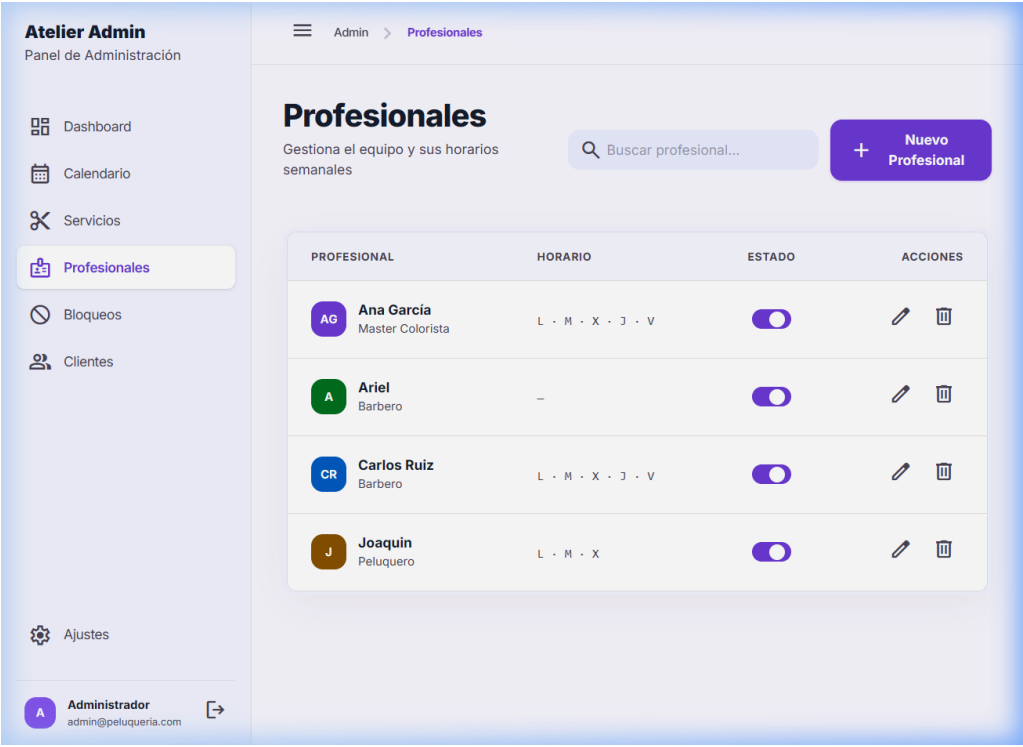


Figura 4.16: Gestión de la plantilla de profesionales.

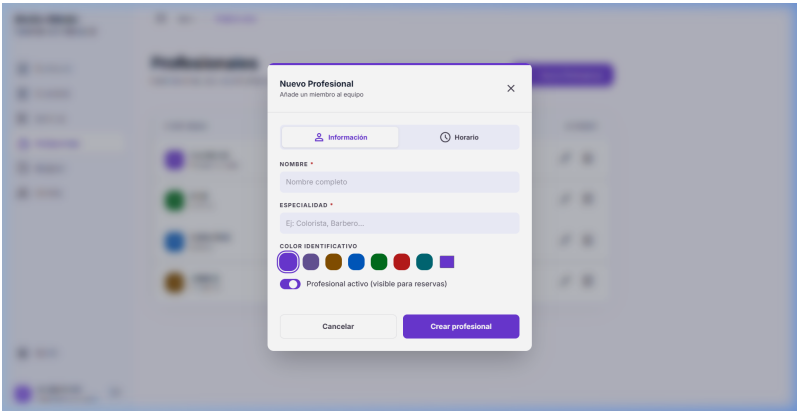


Figura 4.17: Configuración de un nuevo profesional y sus horarios.

4.2.5. Gestión de Bloqueos

Permite definir periodos de inactividad (Figura 4.18) mediante un formulario dedicado (Figura 4.19).



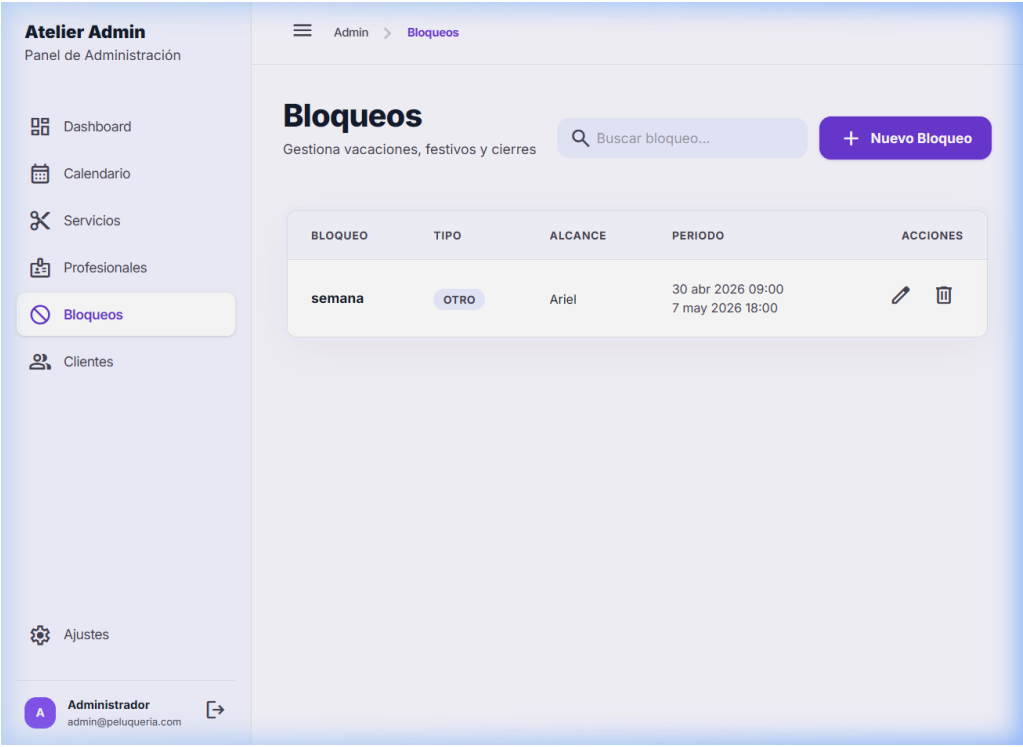


Figura 4.18: Pantalla de configuración de bloqueos horarios y vacaciones.

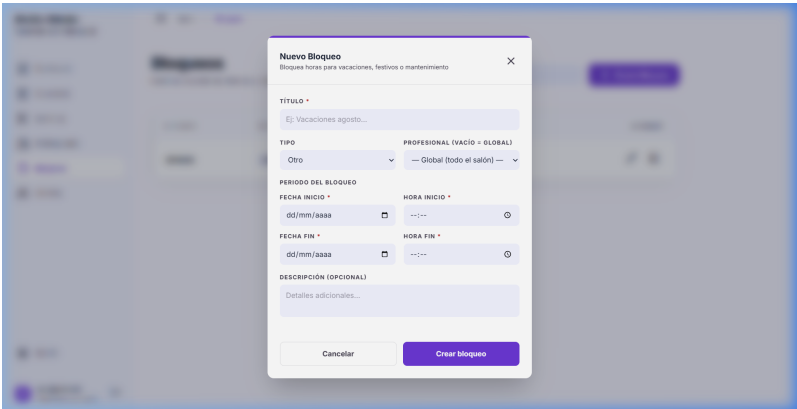


Figura 4.19: Formulario de creación de bloqueo de agenda.

4.2.6. Gestión de Clientes y Notas Técnicas

El administrador puede acceder al historial detallado de cada cliente (Figura 4.20) y gestionar sus notas técnicas (Figura ??), permitiendo la creación de nuevas anotaciones (Figura 4.21).

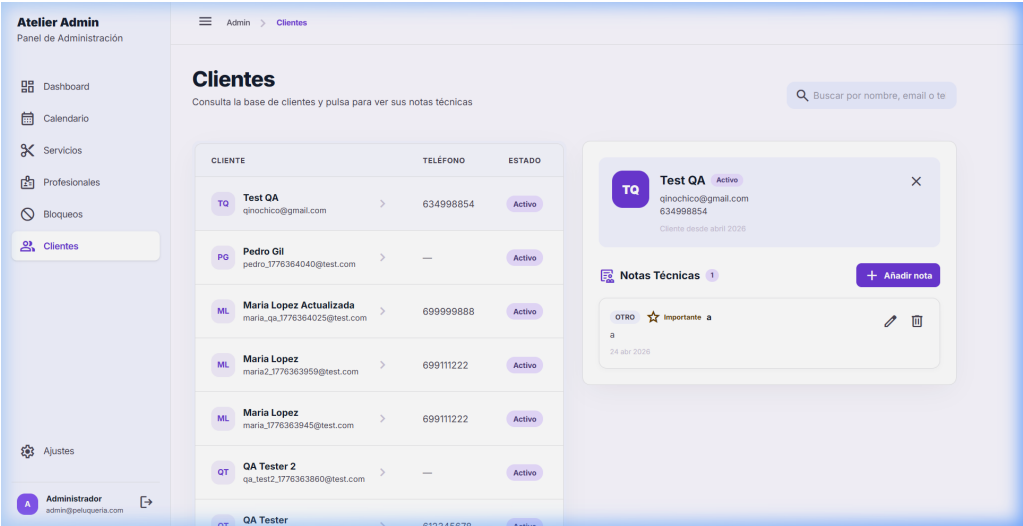


Figura 4.20: Detalle de la ficha de un cliente.

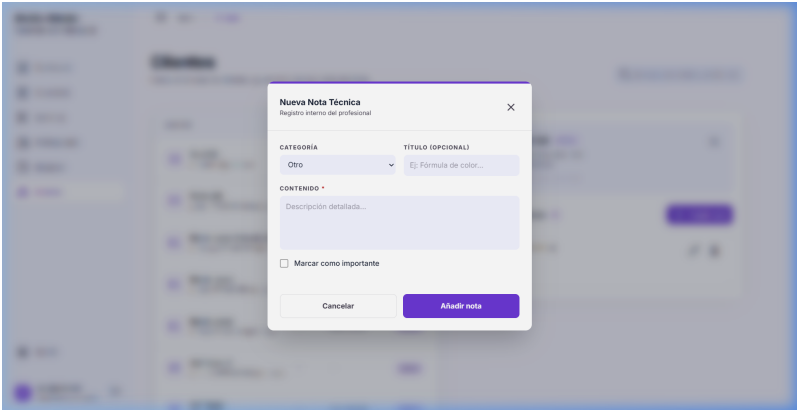


Figura 4.21: Formulario para añadir una nueva nota técnica al expediente del cliente.

4.2.7. Configuración del Negocio

Ajustes globales del sistema, horarios generales y políticas de reserva (Figura 4.22).

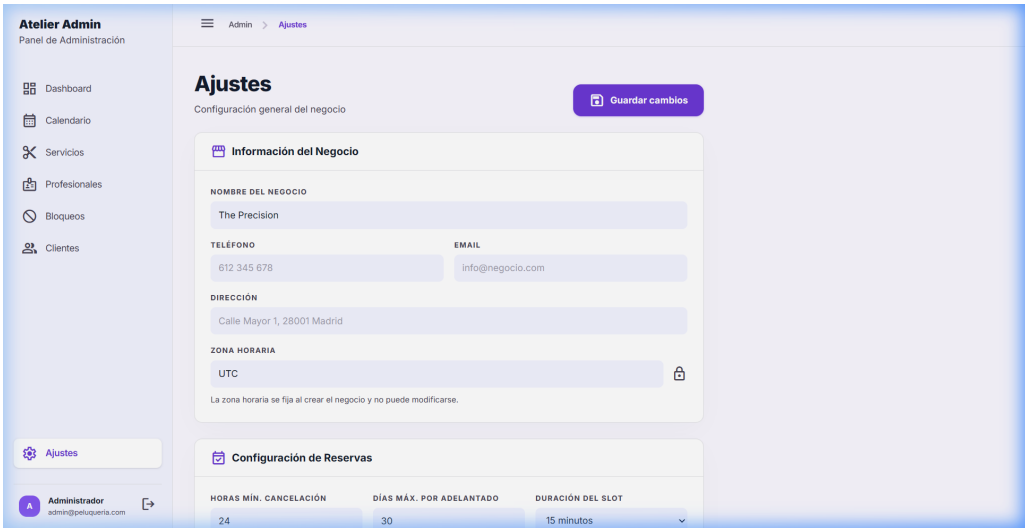


Figura 4.22: Configuración general del establecimiento.

---

## Manual de Despliegue

---

Este capítulo describe los requisitos y pasos necesarios para instalar y ejecutar el sistema en un entorno local de desarrollo, así como las consideraciones para un despliegue en producción.

### 5.1– Requisitos Previos

El cuadro 5.1 lista el software necesario para ejecutar el sistema.

| Software | Versión mínima | Propósito                                                        |
|----------|----------------|------------------------------------------------------------------|
| Node.js  | 18.x           | Entorno de ejecución del servidor y herramientas de construcción |
| npm      | 9.x            | Gestor de paquetes (incluido con Node.js)                        |
| MongoDB  | 6.x            | Base de datos (local o servicio en la nube como MongoDB Atlas)   |
| Git      | 2.x            | Control de versiones y clonado del repositorio                   |

Cuadro 5.1: Requisitos de software

### 5.2– Instalación en Entorno Local

#### 5.2.1. Clonado del Repositorio

```
git clone <URL_DEL_REPOSITORIO> mern-peluqueria
cd mern-peluqueria
```

#### 5.2.2. Configuración del Backend

Desde la raíz del proyecto, acceder al directorio del servidor e instalar las dependencias:

```
cd backend
npm install
```

Crear un archivo `.env` en el directorio `backend/` con las siguientes variables de entorno:

```
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://localhost:27017/mern-peluqueria
JWT_SECRET=<clave_secreta_de_al_menos_32_caracteres>
JWT_EXPIRES_IN=30d
FRONTEND_URL=http://localhost:5173
ADMIN_EMAIL=admin@peluqueria.com
ADMIN_PASSWORD=admin123
```

La variable `JWT_SECRET` debe contener una cadena aleatoria de al menos 32 caracteres. El sistema valida esta longitud al arrancar y no permite iniciar con un secreto insuficiente. Las variables `ADMIN_EMAIL` y `ADMIN_PASSWORD` se utilizan para crear automáticamente el primer usuario administrador si no existe.

### 5.2.3. Configuración del Frontend

Acceder al directorio del cliente e instalar las dependencias:

```
cd frontend
npm install
```

Opcionalmente, crear un archivo `.env` en el directorio `frontend/` para sobrescribir la URL de la API:

```
VITE_API_URL=http://localhost:5000/api
```

Si no se define esta variable, el cliente utiliza `http://localhost:5000/api` por defecto.

### 5.2.4. Inicio del Sistema

Se requieren dos terminales para ejecutar el servidor y el cliente de forma simultánea.

#### Terminal 1 — Backend:

```
cd backend
npm run dev
```

El servidor arranca en el puerto configurado (por defecto 5000) en modo desarrollo con recarga automática mediante `nodemon`. Al primer arranque, verifica la conexión con MongoDB, crea el usuario administrador inicial si no existe, e inicializa la configuración global del negocio.

#### Terminal 2 — Frontend:

```
cd frontend
npm run dev
```

Vite inicia un servidor de desarrollo en `http://localhost:5173` con recarga en caliente (*Hot Module Replacement*). La aplicación es accesible desde el navegador en esa dirección.

## 5.3– Despliegue en Producción

El sistema se ha desplegado en producción utilizando servicios gratuitos en la nube: Render (Render, Inc., 2024) para el alojamiento de la aplicación y MongoDB Atlas (MongoDB, Inc., 2024b) para la base de datos. Esta combinación permite mantener el sistema accesible de forma permanente sin coste. El cuadro 5.2 resume la infraestructura utilizada.

| Componente    | Servicio           | Configuración                                                                                                                               |
|---------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend      | Render Static Site | Directorio raíz: <code>frontend</code> , comando de construcción: <code>npm run build</code> , directorio de publicación: <code>dist</code> |
| Backend       | Render Web Service | Directorio raíz: <code>backend</code> , comando de arranque: <code>node src/index.js</code> , entorno <code>Node.js</code>                  |
| Base de datos | MongoDB Atlas      | Cluster compartido (M0 Free), región Europa                                                                                                 |

Cuadro 5.2: Infraestructura de despliegue en producción

### 5.3.1. Configuración de MongoDB Atlas

El primer paso consiste en crear un cluster gratuito en MongoDB Atlas. Se configura un usuario de base de datos con permisos de lectura y escritura, y se añade la dirección IP `0.0.0.0/0` a la lista de acceso de red para permitir conexiones desde los servidores de Render, cuyas direcciones IP son dinámicas. Atlas proporciona una cadena de conexión con el formato:

```
mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net
/<nombre-bd>?retryWrites=true&w=majority
```

### 5.3.2. Despliegue del Backend en Render

Se crea un servicio de tipo *Web Service* en Render vinculado al repositorio de GitHub. Se configura el directorio raíz como `backend`, el comando de construcción como `npm install` y el comando de arranque como `node src/index.js`. Las variables de entorno necesarias se configuran en el panel de Render:

- `MONGODB_URI`: Cadena de conexión de Atlas con el nombre de la base de datos.
- `JWT_SECRET`: Clave secreta de al menos 32 caracteres.
- `FRONTEND_URL`: URL del *frontend* desplegado, para la configuración de CORS.
- `NODE_ENV`: `production`.

Un aspecto relevante es la configuración de `trust proxy` en Express. Render, como la mayoría de plataformas PaaS, sitúa un *proxy* inverso delante del servidor. Sin la directiva `app.set('trust proxy', 1)`, la biblioteca `express-rate-limit` rechaza las peticiones al detectar la cabecera `X-Forwarded-For` sin la configuración de confianza correspondiente.

### 5.3.3. Despliegue del Frontend en Render

Se crea un servicio de tipo *Static Site* en Render vinculado al mismo repositorio. Se configura el directorio raíz como `frontend`, el comando de construcción como `npm install && npm run build` y el directorio de publicación como `dist`. Como variable de entorno se define `VITE_API_URL` con la URL del *backend* desplegado (por ejemplo, `https://nombre-api.onrender.com/api`).

Render asigna automáticamente un certificado SSL y un dominio `.onrender.com` a cada servicio, proporcionando HTTPS sin configuración adicional. Además, Render redespiega automáticamente la aplicación cada vez que se detecta un nuevo *commit* en la rama principal del repositorio.

#### 5.3.4. URLs de Despliegue y Consideraciones de Acceso

- **Backend:** `https://mern-peluqueria-api.onrender.com`
- **Frontend:** `https://mern-peluqueria.onrender.com`

**Importante:** Debido a las limitaciones de la capa gratuita de Render, el servicio de backend puede entrar en estado de hibernación tras un periodo de inactividad. Por ello, es necesario acceder primero a la URL del backend y esperar unos segundos antes de intentar acceder a la aplicación frontend. De lo contrario, la aplicación cliente podría mostrar errores de conexión temporalmente hasta que el backend esté activo.

---

### Conclusiones

---

#### 6.1– Objetivos Alcanzados

El desarrollo de este proyecto ha permitido alcanzar los objetivos planteados en el capítulo 1:

Se ha diseñado e implementado una base de datos en MongoDB con siete colecciones que modelan todas las entidades del dominio: usuarios, profesionales, servicios, citas, bloqueos temporales, notas técnicas y configuración global. Los esquemas de Mongoose garantizan la validación de datos, las restricciones de tipo y formato, y la integridad referencial a nivel de aplicación.

Se ha desarrollado un portal público de reservas funcional que permite a los clientes consultar el catálogo de servicios, visualizar la disponibilidad en tiempo real y realizar reservas de forma autónoma mediante un flujo guiado de tres pasos. La interfaz se adapta a dispositivos móviles gracias al diseño *responsive* con Tailwind CSS.

Se ha implementado un panel de administración completo que proporciona a los gestores del negocio las herramientas necesarias para el control de la agenda, incluyendo un calendario interactivo con vista por profesional basado en FullCalendar, fichas de cliente con notas técnicas categorizadas (CRM), gestión de horarios semanales con descansos, administración de bloqueos temporales, y un *dashboard* con indicadores de rendimiento.

El motor de disponibilidad calcula los huecos libres considerando horarios de cada profesional, descansos, bloqueos activos y citas existentes, garantizando la imposibilidad de solapamiento de citas. Este cálculo tiene en cuenta la zona horaria configurable del negocio y la rejilla de *slots* definida en los ajustes.

#### 6.2– Conocimientos Aplicados

Este proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos en la asignatura de Complementos de Bases de Datos, especialmente en las siguientes áreas:

- Modelado de datos orientado a documentos con MongoDB y Mongoose.
- Diseño de esquemas con subdocumentos embebidos, referencias entre colecciones y propiedades virtuales.
- Definición de índices compuestos para la optimización de consultas frecuentes.
- Validación de datos a nivel de esquema y a nivel de aplicación.
- Implementación de patrones como el *singleton* para la configuración global con caché en memoria.



- Gestión de la integridad referencial en un entorno NoSQL.

### 6.3– Dificultades Encontradas

Durante el desarrollo se han encontrado dificultades que han requerido especial atención:

La gestión de zonas horarias ha supuesto el mayor reto técnico del proyecto. Todas las fechas se almacenan en UTC en MongoDB, pero los cálculos de disponibilidad y la presentación al usuario deben realizarse en la zona horaria local del negocio. Se han desarrollado utilidades específicas con Day.js para garantizar la coherencia en las conversiones, evitando errores sutiles como la asignación de una cita al día incorrecto en los límites de la medianoche.

El cálculo de disponibilidad con duraciones variables ha requerido un diseño cuidadoso. Un servicio de 20 minutos en una rejilla de 30 minutos debe bloquear el *slot* completo para evitar fragmentación de la agenda. La distinción entre duración real y duración operativa resuelve este problema.

La sincronización de la caché del *frontend* con los cambios en el servidor ha exigido implementar un sistema de invalidación por grupos mediante React Query, garantizando que todos los componentes reflejen datos actualizados tras cada mutación.

### 6.4– Líneas de Trabajo Futuro

El sistema desarrollado sienta las bases para futuras ampliaciones:

- Implementación de notificaciones por correo electrónico y SMS para recordar citas a los clientes, utilizando servicios como SendGrid o Twilio.
- Integración con pasarelas de pago (Stripe, Redsys) para permitir el cobro anticipado o la señal de reservas.
- Desarrollo de un módulo de estadísticas avanzadas con gráficos de tendencias de facturación, servicios más demandados y horarios de mayor afluencia.
- Implementación de bloqueos recurrentes automáticos (el campo `esRecurrente` ya está contemplado en el modelo).

### 6.5– Valoración Personal

Este proyecto ha supuesto una experiencia enriquecedora, ya que nos ha permitido abordar un problema real y cercano, desarrollando una solución completa desde el análisis de requisitos hasta la implementación. El hecho de que el sistema esté destinado a ser utilizado en un negocio familiar ha añadido una motivación adicional y ha permitido obtener retroalimentación directa de los usuarios finales durante el desarrollo, validando decisiones de diseño y ajustando funcionalidades a las necesidades reales del día a día del negocio.

---

### Declaración de Uso de Inteligencia Artificial

---

En cumplimiento de la normativa académica vigente, los autores de este trabajo declaramos que se han utilizado herramientas de inteligencia artificial generativa como apoyo durante el desarrollo del proyecto.

Estas herramientas han sido empleadas como asistentes en tareas de redacción, estructuración de contenidos y desarrollo de código. En concreto, podemos destacar el uso de herramientas como Google Stitch para prototipado de interfaces, Claude y ChatGPT para generación de texto y código, y GitHub Copilot para sugerencias de código durante la implementación, e integración con el tablero de Obsidian donde estructuramos las tareas del proyecto.

En todos los casos, los resultados generados han sido revisados, validados y adaptados por los autores para garantizar su corrección y adecuación a los objetivos del proyecto, quienes asumen la responsabilidad de la integridad y originalidad del trabajo presentado. Todo el material ha sido comprendido en su totalidad y puede ser explicado y defendido ante el tribunal evaluador.

Joaquín Borja León  
Ariel Escobar Capilla

Sevilla, Abril de 2026

---

## Referencias

---

- Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON web token (JWT) — RFC 7519*. Obtenido de <https://datatracker.ietf.org/doc/html/rfc7519>
- Karpov, V. (2024). *Mongoose ODM documentation*. Obtenido de <https://mongoosejs.com/docs/>
- Linsley, T. (2024). *TanStack query — powerful asynchronous state management*. Obtenido de <https://tanstack.com/query/latest>
- McDonnell, C. (2024). *Zod — TypeScript-first schema validation with static type inference*. Obtenido de <https://zod.dev/>
- Meta Platforms, Inc. (2024). *React — the library for web and native user interfaces*. Obtenido de <https://react.dev/>
- MongoDB, Inc. (2024a). *MERN stack explained*. Obtenido de <https://www.mongodb.com/resources/languages/mern-stack>
- MongoDB, Inc. (2024b). *Mongodb atlas — cloud database service*. Obtenido de <https://www.mongodb.com/docs/atlas/>
- MongoDB, Inc. (2024c). *Mongodb documentation*. Obtenido de <https://www.mongodb.com/docs/manual/>
- OpenJS Foundation. (2024). *Express.js — fast, unopinionated, minimalist web framework for Node.js*. Obtenido de <https://expressjs.com/>
- OWASP Foundation. (2024). *Rate limiting — OWASP cheat sheet series*. Obtenido de [https://cheatsheetseries.owasp.org/cheatsheets/Denial\\_of\\_Service\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html)
- Render, Inc. (2024). *Render — cloud application hosting for developers*. Obtenido de <https://docs.render.com/>
- Shaw, A. (2024). *FullCalendar — full-sized JavaScript calendar*. Obtenido de <https://fullcalendar.io/>
- Wathan, A. (2024). *Tailwind CSS — rapidly build modern websites without ever leaving your HTML*. Obtenido de <https://tailwindcss.com/>
- You, E. (2024). *Vite — next generation frontend tooling*. Obtenido de <https://vite.dev/>